

Homework 4 - Monte Carlo Tree Search for Orienteering Problem

Francesco Gnan

Introduction

In this homework we want to solve the Orienteering Problem with Time Windows (OPTW) using Monte Carlo Tree Search (MCTS). We will refer to the code in the notebook `FrancescoGnan_MSCAI23_hw4.ipynb`. In particular, we implement the MCTS to search for the global minimum of the OPTW random generated instances: 0, 1, 2 (small size problems).

Theoretical Recalls

MCTS is a method for finding optimal decisions in a given domain by taking random samples in the decision space and building a search tree according to the results. The algorithm works as follows:

- A tree is built in an incremental and asymmetric manner.
- For each iteration of the algorithm, a tree policy is used to find the most urgent node of the current tree. The tree policy attempts to balance considerations of exploration (look in areas that have not been well sampled yet) and exploitation (look in areas which appear to be promising).
- A simulation is then run from the selected node and the search tree updated according to the result.
- This involves the addition of a child node corresponding to the action taken from the selected node, and an update of the statistics of its ancestors.
- Moves are made during this simulation according to some default policy, which in the simplest case is to make uniform random moves.

Our policy will take into account the so-called upper confidence bound (UCB) that any given arm will be optimal. It is defined as:

$$UCT = \bar{X}_j + 2C_p \sqrt{\frac{2 \ln n}{n_j}} \quad (1)$$

$\bar{X}_j$ is the average reward from arm j , n_j is the number of times arm j was played and n is the overall number of plays so far. The reward term $\bar{X}_j$ encourages the exploitation of higher-reward choices, while the right hand term $\sqrt{\frac{2 \ln n}{n_j}}$ encourages the exploration of less-visited choices. As each node is visited, the denominator of the exploration term increases, which decreases its

contribution. On the other hand, if another child of the parent node is visited, the numerator increases and hence the exploration values of unvisited siblings increase. The exploration term ensures that each child has a non- zero probability of selection, which is essential given the random nature of the playouts.

As far as MCTS is concerned, it rests on two fundamental concepts: that the true value of an action may be approximated using random simulation; and that these values may be used efficiently to adjust the policy towards a best-first strategy. The algorithm progressively builds a partial game tree, guided by the results of previous exploration of that tree. The tree is used to estimate the values of moves, with these estimates (particularly those for the most promising moves) becoming more accurate as the tree is built.

In each search iterations there are four steps:

1. **Selection:** Starting at the root node, a child selection policy is recursively applied to descend through the tree until the most urgent expandable node is reached. A node is expandable if it represents a nonterminal state and has unvisited children.
2. **Expansion:** One or more child nodes are added to expand the tree, according to the available actions.
3. **Simulation:** A simulation is run from the new node(s) according to the default policy to produce an outcome.
4. **Backpropagation:** The simulation result is backpropagated through the selected nodes to update their statistics.

As we have already said, every time a node (action) is to be selected within the existing tree, the choice may be modelled as an independent multi-armed bandit problem. A child node j is selected to maximise Equation 1. Finally, the return value of the overall search is given by

$$\operatorname{argmax}_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c \sqrt{\frac{2 \ln N(v)}{N(v')}} \quad (2)$$

which will give the action a that leads to the child with the highest reward. v indicates a given node, $Q(v)$ is the total simulation reward and $N(v)$ indicates the visit count.

Implementation

We complete the following functions in the provided code.

Tree policy

This function is responsible for selecting the next node to explore in the search tree.

```
def treePolicy(self, node):
    while node is not None and not node.terminal:
        if not node.fully_expanded:
            return self.expand(node)
        else:
            node = self.selectBestChild(node)

    return node
```

It takes as input the current node in the search tree from which the algorithm decides where to go next. If this node has not been fully expanded, it calls the `expand()` function to generate child nodes for the current node. If it is fully expanded, instead, it calls the `selectBestChild()` function to select the best child node based on Equation 2, that balances exploration and exploitation. Finally, it returns the selected node, i.e. either a new expanded node or the best child node.

Select best child

This function selects the best child node from a given node based on the exploration-exploitation strategy in Equation 2.

```
def selectBestChild(self, node):
    best_score = 0
    best_child = None
    c = 1 / math.sqrt(2)

    for child in node.expandables:
        child = node.expanded.get(child)
        if child is not None:
            Q_over_N = child.current_reward / (child.num_of_visit)
            sqrt = c * np.sqrt( 2 * np.log(node.num_of_visit) / (child.
                num_of_visit))

            child_score = Q_over_N + sqrt

            if child_score > best_score:
                best_score = child_score
                best_child = child

    return best_child
```

Here we set the constant c to $\frac{1}{\sqrt{2}}$ as suggested in the paper “MCTS survey”.

Backup

This function updates information in the nodes of the search tree after a simulation has been performed from a specific node. In other words, it backpropagates the results of the simulation back up the tree.

```
def backup(self, node, reward_esteem):
    while node is not None:
        node.num_of_visit += 1
        node.estimated_reward += reward_esteem
        node = node.parent
```

After updating the information of the current node, the loop moves to the parent of the current node. This step continues until the loop reaches the root of the tree. This backpropagated information is crucial for guiding the selection and expansion steps in subsequent iterations of the MCTS algorithm, helping the algorithm focus on more promising areas of the decision space.

Results

In this section we report, for each instance, the best solution found by the algorithm. They are shown in Figure 1. We clearly see that, in order to be feasible, the best solutions connect a small subset of the entire number of blue dots in the space.

Conclusion

In conclusion, we have applied the MCTS algorithm to solve the Orienteering Problem with Time Window. We have completed some missing function within the provided code and at the end we have tested it on three different small-sized instances. We have shown the results in Figure 1.

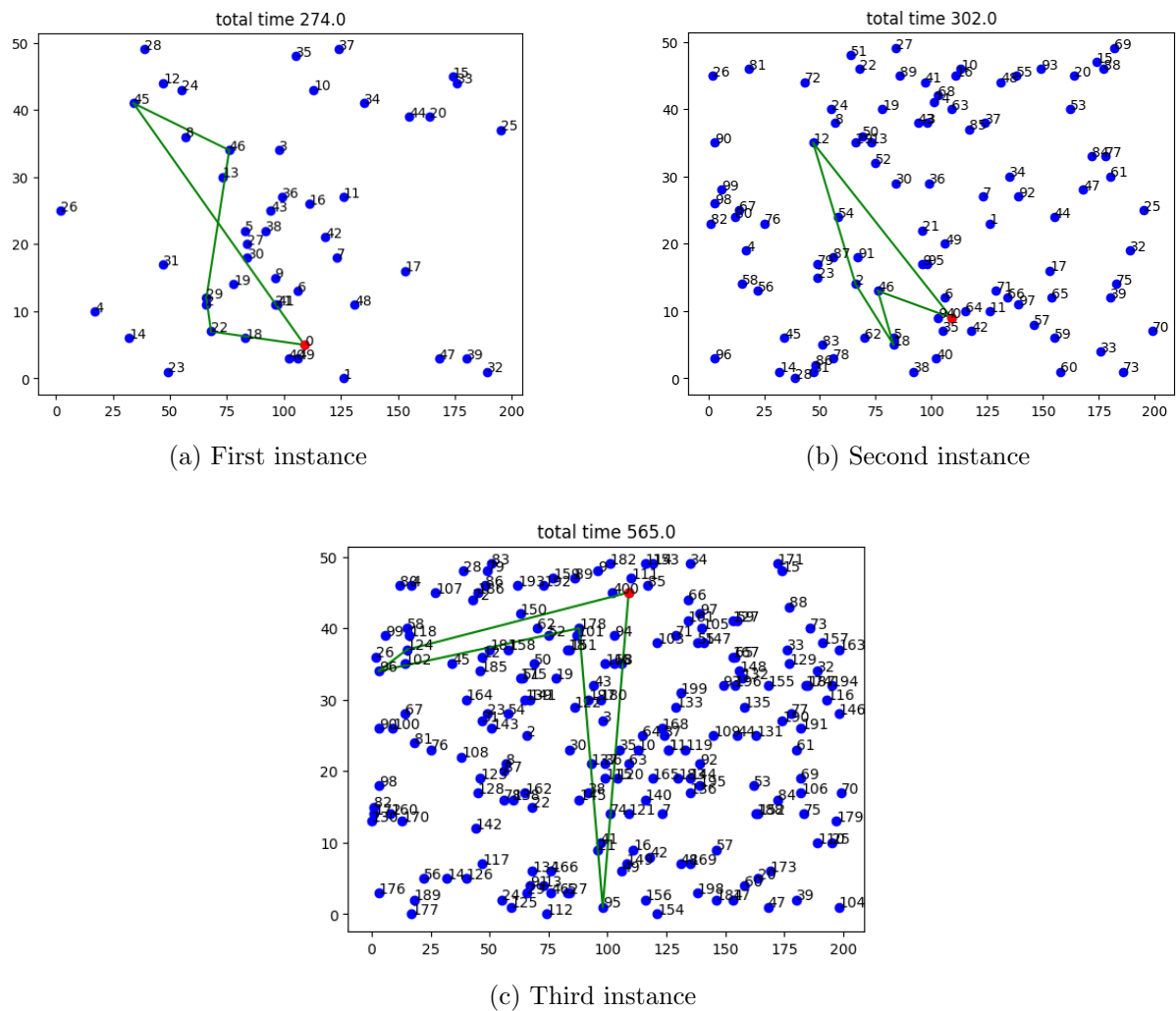


Figure 1: Best solutions